ARCHITECTURE

Architecture -- Containers

Purpose

Generic, reusable Go module for container orchestration, health checking, lifecycle management, service discovery, and remote distribution. Supports Docker, Podman, and Kubernetes runtimes with SSH-based multi-host container scheduling, tunnel management, and volume sharing.

Structure

```
pkg/
  runtime/       Container runtime abstraction (Docker/Podman/K8s/
RemoteRuntime)
  compose/       Docker Compose orchestration (batch operations
grouped by file/profile)
  health/        Health checking: TCP, HTTP, gRPC, Custom with
retry
  endpoint/      Service endpoint configuration (builder pattern)
  lifecycle/     Advanced lifecycle: lazy boot, idle shutdown,
concurrency semaphores
  monitor/       Resource monitoring: CPU/memory/disk per-
container, cluster snapshots
  event/         Event bus for 20 lifecycle event types (publish/
subscribe)
  discovery/     Service discovery: TCP port probe and DNS-based
  logging/       Pluggable logging abstraction (slog adapter
included)
  metrics/       Prometheus-compatible metrics collection
  boot/          High-level BootManager composing all subsystems
  remote/        Remote host management, SSH executor, connection
pooling
  scheduler/     Resource-aware container scheduling (5
strategies)
  network/       SSH tunnel management, port allocation, overlay
networks
```

```
  volume/        Remote volume management (SSHFS/NFS/rsync)
  envconfig/     Environment-variable-based configuration for
remote hosts
  distribution/  Distribution orchestrator: schedule, deploy,
failover
  orchestrator/  Service orchestrator with auto-discovery and
remote support
  ctop/          Real-time container monitoring TUI (top/htop-
style)
cmd/
  ctop/          CLI entry point for container monitoring
```

Key Components

- **`runtime.ContainerRuntime`** -- Interface for Docker/Podman/K8s with auto-detection
- **`boot.BootManager`** -- High-level orchestrator composing runtime, compose, health, discovery, events, metrics, and logging
- **`distribution.Distributor`** -- Facade composing scheduler, remote executor, tunnel manager, and volume manager for multi-host deployment
- **`scheduler.Scheduler`** -- 5 placement strategies: resource_aware, round_robin, affinity, spread, bin_pack
- **`lifecycle.LifecycleManager`** -- Lazy boot (start on first acquire), idle shutdown, concurrency semaphores
- **`ctop.Display`** -- Interactive TUI for real-time container monitoring across local and remote hosts

Data Flow

```
BootManager.BootAll(ctx) -> for each endpoint:
    compose.Up(file, service) -> health.Check(endpoint) ->
event.Publish(ServiceStarted)
        |                                |
    (remote) distribution.Distribute() ->
scheduler.Schedule(requirements, hosts)
        |                                          |
    remote.SSHExecutor.Execute()
network.TunnelManager.Create()
        |                                          |
    volume.VolumeManager.Mount()
monitor.ResourceMonitor.Collect()
```

Dependencies

- `github.com/gorilla/websocket` -- WebSocket for event streaming
- `github.com/stretchr/testify` -- Test assertions

Testing Strategy

Unit tests with `testify` and race detection. Integration tests require a container runtime (Docker or Podman). Tests cover runtime auto-detection, health check retries, compose orchestration, event publishing, lifecycle management, scheduler strategy selection, and SSH tunnel creation.

GPU-Aware Scheduling

See `docs/gpu-scheduling.md`. GPU support is fully additive: callers that ignore it get the exact pre-2026-04 behaviour.